A Poissonized Evaluator for the Multinomial Inversion Series

Exact de-Poissonization by a discrete Cauchy integral on the saddlepoint circle

Abstract

Every term of the inversion series is a constrained transform: the coefficient of u^N in a product of K per-category polynomials. The existing evaluators obtain it by multiplying the polynomials together and cutting the running product back to degree N after every category; in the FFT tree that truncation fixes the working size at $M \approx 2$–$4N$ and costs a full descent, leaf convolution and ascent per category, driven by a precomputed plan of merge factors whose size grows like $KM \log M$. This document describes an evaluator that drops the constraint instead of enforcing it. The categories become independent, each polynomial is evaluated on its own at Q equispaced points of a circle, the K evaluations are multiplied pointwise, and the coefficient is recovered *exactly*, up to aliasing, by a Q-point trapezoid rule for Cauchy's integral. With the circle's radius at the saddlepoint of the tilted total count — the same tilt the gamma search and the FFT tree already use — the aliasing is suppressed like the tails of that count, and two facts make the choice of Q certified rather than heuristic: the aliasing at frequency zero bounds it for every term of the series, and that frequency-zero aliasing can be measured exactly by comparing Q against $2Q$. Per frequency the work is one fold of $N + 1$ coefficients and one length-Q FFT per category, with Q of seven to ten standard deviations of the tilted count. It agrees with the FFT tree to 10^{-15}–2.3×10^{-12} on the p-value at a fixed term budget. Measured on an Apple M5, one term costs about a third as much in the deep tail when Q equals the tree's working size (2.1–3.6× faster), 4–20× less when Q is a half or a quarter of it, which happens at many categories, and 4–22× less away from the tail; the plan is 30–140× smaller (4 MB against 588 MB at $N = 10^4$, $K = 50$).

The same idea – drop the constraint, work with independent categories – is already used elsewhere in this codebase to select the tilt γ itself, and we derive that construction in full: the Poissonized log-MGF is convex in γ, its derivatives come out of the same one-pass moment computation the tilt ν is already solved for by an application of the envelope theorem, and a safeguarded Newton iteration replaces a 128-point grid at 13–25× less time per instance. We explain why the same *approximate* (never de-Poissonized) construction that is exact enough for γ is not accurate enough for the series terms themselves. Three further, evaluator-independent changes round out the implementation: a persistent thread pool replaces per-chunk thread creation in `series()` (2× at the smallest instances); the adaptive stopping rule for `extrapolate` ≥ 2 combines the pre-existing term-ratio rule with a check on a shallow, median-smoothed extrapolation's trailing-window stability, verified stable to 10^{-9}–10^{-13} under a one-ulp perturbation of every term, across both evaluators and all thread counts tried; and locating the statistic's range before the series can run – previously an $O(KN^2)$ dynamic program – is now $O(K)$ for the power-divergence family (an exact vertex argument for the minimum, a closed-form bound for the maximum) and $O(K \log K)$ for the probability-mass statistic (an exact maximum via White and Hendy (2010)'s multinomial-mode algorithm), cutting setup time by up to 4–5× at $N = 10^4$.

Contents

1 What each series term is **2**

2 The truncated product, and what it costs **3**

3 Poissonization **3**

4 Exact de-Poissonization: the discrete Cauchy formula **4**

5 Aliasing: one bound for every term, and a certificate for it **5**

6 Why Q is large exactly in the deep tail **6**

7 Why the approximate de-Poissonization is not enough **7**

8 An aside: selecting γ by the same Poissonisation **7**

9 The evaluator **9**

10 Verification **11**

11 The statistic's interval, without the $O(KN^2)$ recursion **11**

12 Measured speedup **13**
12.1 Per frequency 13
12.2 Memory and plan construction 15
12.3 End to end 15

13 Robust stopping for `extrapolate` ≥ 2 **16**

14 Summary **16**

1 What each series term is

We keep the notation of the companion documents. $X \sim \text{Multinomial}(N, \mathbf{p})$ over K categories, $S = \sum_k a_k(X_k)$ is the centred statistic, γ is the tilt chosen by `fill_gamma`, $z_n = \gamma + inT$, and $M_S(z) = \mathbb{E}[e^{-zS}]$. The series accumulates $t_n = M_S(z_n)\,\mathsf{sinc}(z_n)$ with $\mathsf{sinc}(z) = \mathbb{E}[e^{-zU}]$, $U \sim \text{Unif}[0, W]$, $W = \pi/T$; see `error_bounds.tex` for why $p = \frac{1}{2}\sum_{n\in\mathbb{Z}} t_n$. Everything below concerns computing $M_S(z_n)$ for one frequency n.

Write $p_k(i) = p_k^i/i!$, so that $\log p_k(i) = i \log p_k - \log i!$ as in `rates.tex`. After `fill_cells`, cell `matrix[k*(N+1)+i]` stores

$$\ell_k(i) \;:=\; \texttt{log_prob} \;=\; \log p_k(i) - \gamma a_k(i), \qquad \theta_k(i) \;:=\; \texttt{reward} \;=\; T\, a_k(i),$$

so that $p_k(i)\, e^{-z_n a_k(i)} = e^{\ell_k(i)} e^{-in\theta_k(i)}$. Define the per-category polynomials

$$\phi_{k,n}(u) \;=\; \sum_{i=0}^{N} e^{\ell_k(i)}\, e^{-in\theta_k(i)}\, u^i.$$

Proposition 1 (The term as a coefficient)**.** *For every n,*

$$M_S(z_n) \;=\; N!\,[u^N] \prod_{k=1}^{K} \phi_{k,n}(u).$$

Proof. $P(X=x) = N!\prod_k p_k(x_k)$ on $\{x \in \mathbb{Z}^K_{\geq 0} : \sum_k x_k = N\}$, so $M_S(z_n) = N! \sum_{\sum x_k = N} \prod_k e^{\ell_k(x_k) - in\theta_k(x_k)}$. Expanding the product of the $\phi_{k,n}$, the coefficient of u^N collects exactly these products; the bound $x_k \leq N$ is automatic. □

2 The truncated product, and what it costs

The reference evaluator (`eval_fourier_thread`) and the FFT tree (`fourier_fft_precompute.h`, analysed in `rates.tex`) both build the product one category at a time and discard every coefficient above degree N after each step — those never reach $[u^N]$. Truncation keeps every intermediate polynomial at degree $\leq N$, so each step is a product of two polynomials of degree $\leq N$ and fits in a ring of size $M = 2 \cdot 2^{\lceil \log_2(N+1) \rceil}$.

That truncation is also what makes each step expensive. The tree works modulo $x^M - 1$ by recursive ring splitting: a descent to width 4, direct convolutions at the leaves, and an ascent that reconstructs the product before it can be truncated. Counting call sites in `fourier_fft_count_calls`,

$$\text{calls}(M) \;=\; \tfrac{3}{2} M \,(\log_2 M - 2) \;+\; 4M \tag{1}$$

merges per category, each applying a stored magnitude factor and a complex rotation to all SIMD lanes; the precompute plan stores $K \cdot \text{calls}(M)$ factors of 16 bytes. At $N = 1000$ that is $M = 2048$ and 35,840 merges per category; at $N = 10^4$ it is $M = 32{,}768$, 770,048 merges, and a 588 MB plan for $K = 50$.

3 Poissonization

The constraint $\sum_k x_k = N$ is what couples the categories. The classical way to remove it is to make the counts independent Poisson variables and condition on their total afterwards: if $Y_k \sim \text{Poisson}(\lambda p_k)$ independently, then $(Y_1, \ldots, Y_K) \mid \sum_k Y_k = N$ is Multinomial$(N, \mathbf{p})$ (Levin, 1981). In generating-function language, the product $\prod_k \phi_{k,n}$ is the unconstrained (Poissonized) object, and $[u^N]$ is the conditioning; the whole difficulty lies in undoing the Poissonization, the *de-Poissonization* (Jacquet and Szpankowski, 1998).

The tilt. For $\nu \in \mathbb{R}$ and $\rho = e^\nu$, define on $\{0, \ldots, N\}$

$$w_k(i) = e^{\ell_k(i) + \nu i}, \qquad Z_k = \sum_{i=0}^{N} w_k(i), \qquad \pi_k(i) = \frac{w_k(i)}{Z_k},$$

and let $Y_k \sim \pi_k$ be independent with total $T = \sum_k Y_k$. Then

$$\prod_k \phi_{k,0}(\rho u) \;=\; \Big(\prod_k Z_k\Big)\, \mathbb{E}\big[u^T\big], \qquad \text{so} \qquad [u^N] \prod_k \phi_{k,0}(u) \;=\; \rho^{-N} \Big(\prod_k Z_k\Big) P(T = N). \tag{2}$$

We fix ν by $\mathbb{E}[T] = N$. The solution is unique, since $\frac{d}{d\nu}\mathbb{E}[T] = \operatorname{Var} T > 0$, and it is precisely the saddlepoint: it minimizes $\rho^{-N} \prod_k Z_k$ over ρ. It is the same tilt that `rates.tex` derives for the FFT tree (`l_rate`) and that the Newton gamma search solves for in `gamma_search.h`; the evaluator re-solves it on the transformed cells in `poisson_tilt`, warm-started from `mult->l_rate`, so it does not depend on which search produced γ. Throughout,

$$V \;:=\; \operatorname{Var} T \;=\; \sum_k \operatorname{Var} Y_k$$

is the variance of the tilted total count, the `v_i` accumulator of `gamma_moments`.

4 Exact de-Poissonization: the discrete Cauchy formula

Proposition 2 (Trapezoid rule for a Taylor coefficient)**.** *Let* $F(u) = \sum_{j=0}^{D} c_j u^j$, $\rho > 0$, $Q \geq 1$ *and* $\omega = e^{2\pi i/Q}$. *Then*

$$\frac{1}{Q}\sum_{q=0}^{Q-1} \omega^{-qN} \rho^{-N} F(\rho\,\omega^q) \;=\; \sum_{\ell\in\mathbb{Z}} c_{N+\ell Q}\, \rho^{\ell Q},$$

with $c_j := 0$ *outside* $[0, D]$. *In particular the left side equals* c_N *exactly whenever* $Q > \max(N, D - N)$.

Proof. Substituting the expansion, $\frac{1}{Q}\sum_q \omega^{q(j-N)}$ is 1 if $j \equiv N \pmod{Q}$ and 0 otherwise. A term with $\ell \geq 1$ needs $N + \ell Q \leq D$, i.e. $Q \leq D - N$; one with $\ell \leq -1$ needs $N + \ell Q \geq 0$, i.e. $Q \leq N$. □

This is the discretized Cauchy integral over the circle $|u| = \rho$, the standard device for computing Taylor coefficients numerically (Lyness and Moler, 1967; Bornemann, 2011); for analytic periodic integrands the trapezoid rule converges exponentially (Trefethen and Weideman, 2014), and here the integrand is a trigonometric polynomial, so the only error is the aliasing written out explicitly above.

Applied to $F = \prod_k \phi_{k,n}$, which has degree $D = KN$, Propositions 1 and 2 give

$$M_S(z_n) + A_n(Q) \;=\; \underbrace{N!\,\rho^{-N}\prod_{k} Z_k}_{\texttt{resN_exp}} \cdot \underbrace{\frac{1}{Q}\sum_{q=0}^{Q-1} \omega^{-qN} \prod_{k=1}^{K} \hat{\phi}_{k,n}(q)}_{\texttt{unit}}, \qquad \hat{\phi}_{k,n}(q) = \sum_{i=0}^{N} \pi_k(i)\, e^{-in\theta_k(i)}\, \omega^{qi}, \tag{3}$$

where the aliasing is

$$A_n(Q) \;=\; N!\,\rho^{-N} \sum_{\ell\neq 0} c_{N+\ell Q}(n)\, \rho^{N+\ell Q}, \qquad c_j(n) := [u^j] \prod_k \phi_{k,n}(u),$$

and vanishes identically once $Q > \max(N, (K-1)N)$.

Three features of (3) are what make it cheap.

- **No coupling.** Each $\hat{\phi}_{k,n}$ is computed from category k alone, and the categories meet only in a pointwise product over q.
- **One FFT per category.** Folding the coefficients modulo Q, $\hat{\phi}_{k,n}(q) = \sum_{b<Q} B_{k,n}(b)\, \omega^{qb}$ with $B_{k,n}(b) = \sum_{i\equiv b\,(Q)} \pi_k(i) e^{-in\theta_k(i)}$, so all Q values come from a single length-Q transform of the folded vector.
- **Modulus at most one.** Since $\sum_i \pi_k(i) = 1$, every $|\hat{\phi}_{k,n}(q)| \leq 1$ and so is every partial product. Everything that could overflow or underflow — $N!$, ρ^{-N} and the Z_k — sits in the frequency-independent factor, which the plan computes once in quad precision, exactly as the precompute plan does for its own `resN_exp`.

Remark 1 (Conditioning of the extraction)**.** *At* $n = 0$ *the terms of the* q*-sum have modulus at most one while their mean is* $P(T = N)$, *which by the local central limit theorem is about* $1/\sqrt{2\pi V}$. *The extraction therefore loses about* $\log_{10}\sqrt{2\pi V}$ *digits to cancellation:* 3.0 *digits at* $N = 1000$, $K = 7$, *and* 3.8 *digits at* $N = 10^4$, $K = 3$. *This is the sense in which the saddlepoint radius is the natural one: it minimizes* $\rho^{-N}\prod_k Z_k$, *the modulus bound the cancellation has to reduce (Bornemann, 2011).*

5 Aliasing: one bound for every term, and a certificate for it

Lemma 1 (Frequency zero dominates)**.** *For all n and j, $|c_j(n)| \le c_j(0)$. Consequently, with*

$$\alpha(Q) \;:=\; \sum_{\ell \neq 0} \frac{P(T = N + \ell Q)}{P(T = N)},$$

the aliasing satisfies $|A_n(Q)| \le A_0(Q) = \alpha(Q)\, M_S(\gamma)$, and the aliasing in the series terms satisfies $|\Delta t_n| \le \alpha(Q)\, t_0$ for every n.

Proof. $c_j(n) = \sum_{i_1+\cdots+i_K=j} \prod_k e^{\ell_k(i_k)} e^{-in\theta_k(i_k)}$, and $c_j(0)$ is the same sum of moduli, which gives the first claim by the triangle inequality. By (2), $c_j(0)\rho^j = \prod_k Z_k\, P(T = j)$, so $A_0(Q) = N!\rho^{-N} \prod_k Z_k \sum_{\ell\neq 0} P(T = N + \ell Q)$ while $M_S(\gamma) = N!\rho^{-N} \prod_k Z_k\, P(T = N)$. Finally $|\,\mathsf{sinc}(z_n)| = |\mathbb{E}\, e^{-z_n U}| \le \mathbb{E}\, e^{-\gamma U} = \mathsf{sinc}(\gamma)$. □

So a single number, $\alpha(Q)$, bounds the aliasing of every term on the absolute scale of the first one. The bound is not merely valid but attained: in every case we measured, the largest term-by-term aliasing over the first 256 frequencies equals $\alpha(Q)$ to two or three significant digits (Table 1), i.e. the worst term is t_0 itself.

The certificate. $\alpha(Q)$ is measurable without knowing $P(T = N)$. At $n = 0$ the unit in (3) is $\sigma(Q) := \sum_\ell P(T = N + \ell Q)$, since $\prod_k \hat\phi_{k,0}(q) = \mathbb{E}[\omega^{qT}]$. Comparing Q with $2Q$ removes the even multiples:

Proposition 3 (What comparing Q with $2Q$ measures)**.** *With $D(Q) := \big(\sigma(Q) - \sigma(2Q)\big)/\sigma(2Q)$,*

$$D(Q) \;=\; \frac{\alpha(Q) - \alpha(2Q)}{1 + \alpha(2Q)}.$$

Hence $D(Q) \le \alpha(Q)$ always. If moreover $\alpha(2Q) \le \alpha(Q)^2$, then $\alpha(Q) \le D(Q)\,\frac{1+\alpha(Q)^2}{1-\alpha(Q)} = D(Q)\,\big(1 + O(D(Q))\big)$.

Proof. $\sigma(Q) - \sigma(2Q) = \sum_{\ell \text{ odd}} P(T = N + \ell Q) = P(T = N)\,\big(\alpha(Q) - \alpha(2Q)\big)$ and $\sigma(2Q) = P(T = N)\,(1 + \alpha(2Q))$. For the second part, $\alpha(Q) - \alpha(Q)^2 \le \alpha(Q) - \alpha(2Q) = D(Q)(1 + \alpha(2Q)) \le D(Q)(1 + \alpha(Q)^2)$. □

The condition $\alpha(2Q) \le \alpha(Q)^2$ says that doubling Q at least squares the tail suppression. It holds whenever the law of T is log-concave: then $d \mapsto \log P(T = N + d)/P(T = N)$ is concave and vanishes at $d = 0$, so $P(T = N + 2\ell Q)/P(T = N) \le \big(P(T = N + \ell Q)/P(T = N)\big)^2$ for each ℓ, and summing over $\ell \neq 0$ gives the claim.

Proposition 4 (Log-concavity for the probability-mass statistic)**.** *For the probability-mass statistic with $\gamma \le 1$, every π_k is log-concave on $\{0, \dots, N\}$, and so is the law of T.*

Proof. Here $a_k(i) = \log p_k(i) + \text{const}_k$, so $\ell_k(i) + \nu i = (1-\gamma)(i \log p_k - \log i!) + \nu i + \text{const}$. Its second difference in i is $-(1-\gamma)\log\frac{i+1}{i} \le 0$, because linear terms have none and $\Delta^2(-\log i!) = -\log\frac{i+1}{i}$. The support is an interval, so π_k is log-concave, and convolutions of log-concave sequences without internal zeros are log-concave (Hoggar, 1974). □

Remark 2 (The log-likelihood ratio)**.** *For LLR,* $a_k(i) = -i \log \frac{i}{Np_k} + \text{const}_k$ *and* $\ell_k(i) + \nu i = -\log i! + \gamma\, i \log i +$ (*affine in* i). *Since* $\Delta^2(-\log i!) = -\log \frac{i+1}{i} = -\frac{1}{i} + \frac{1}{2i^2} + O(i^{-3})$ *and* $\Delta^2(i \log i) = \frac{1}{i} + O(i^{-3})$*, its second difference is* $-(1-\gamma)/i + 1/(2i^2) + O(i^{-3})$*. It is log-concave only for* $i \gtrsim 1/(2(1-\gamma))$ *— about* $i > 125$ *at* $\gamma = 0.996$ *— so Proposition 4 does not cover the deep tail. For LLR the certificate rests on measurement instead, and Table 1 shows it matching the actual aliasing there just as closely.*

Choosing Q. `build_poisson_plan` proceeds as follows.

1. Start at the smallest power of two $Q \geq 4\sqrt{V}$ (and ≥ 16). If $Q > \max(N, (K-1)N)$ the formula is exact and we stop.
2. Compute $D(Q)$ from $\sigma(Q)$ and $\sigma(2Q)$, in a separate scalar code path (`poisson_s0`) that shares nothing with the vectorized evaluator.
3. Accept Q if $D(Q) \leq 10^{-10}$, or if $D(Q) < 10^{-6}$ has stopped decreasing by at least a factor ten — the rounding floor, where aliasing is already gone. The smallness condition matters: below about two standard deviations D saturates near one and also stops decreasing.
4. Otherwise double Q and repeat, stopping as soon as $Q > \max(N, (K-1)N)$; Q is capped at 2^{20} with a warning.

Starting low matters because Q moves in factors of two and the certificate can only raise it. The tolerance 10^{-10} applies to the *terms*, which is conservative for p: aliasing concentrates in the lowest frequencies and largely cancels in the sum, and its measured effect on the partial sums is two to six orders of magnitude below $D(Q)$ (last column of Table 1).

6 Why Q is large exactly in the deep tail

Since Q scales like $\sqrt{V}$ while M scales like N, everything turns on how broad the tilted per-category laws are.

Away from the tail γ is small, the tilt is mild, and π_k is close to a truncated Poisson law: at $\gamma = 0$, $\pi_k(i) \propto (p_k\rho)^i/i!$, the constraint $\mathbb{E}[T] = N$ gives $\rho = N$, and $V = N$ exactly. Measured with x_0 at the mode: $\sqrt{V} = 10.8$ at $N = 100$, $K = 20$ ($\gamma = 0.14$), and $\sqrt{V} = 33$–37 at $N = 1000$. There $Q \approx 10\sqrt{N} \ll M$, and the fold of $N+1$ coefficients, not the FFT, dominates the cost.

In the deep tail $\gamma \to 1$ and the tilt flattens the per-category laws. For the probability-mass statistic the effect is exact: at $\gamma = 1$ the factor $(1-\gamma)$ in the proof of Proposition 4 vanishes, $\pi_k(i) \propto e^{\nu i}$, and every category carries the *same* truncated geometric law, whatever p_k is, with mean N/K. A geometric law has standard deviation comparable to its mean, so $V \approx K(N/K)^2$ and $\sqrt{V} \approx N/\sqrt{K}$. For LLR, Stirling gives $e^{\ell_k(i)+\nu i} \propto e^{(1+\nu')i}/\sqrt{2\pi i}$ at $\gamma = 1$ — geometric up to the $\sqrt{i}$ factor — and the same picture holds. Measured at $N = 10^4$ with uniform x_0: $\sqrt{V} = 2471$, 2362, 1989, 1558, 1060 for $K = 3, 5, 10, 20, 50$ (LLR), against $N/\sqrt{K} = 5774$, 4472, 3162, 2236, 1414; the ratio 0.43–0.75 reflects $\gamma < 1$ and the truncation at N, which bites for small K.

So in the regime this codebase exists for, Q grows linearly in N and the advantage over the tree is a constant factor rather than an asymptotic one; it is largest for many categories, where $\sqrt{V}$ shrinks like $1/\sqrt{K}$ while M does not change.

instance	$\sqrt{V}$	Q	$Q/\sqrt{V}$	certificate $D(Q)$	$\max_{n<256} \lvert\Delta t_n\rvert/t_0$	partial-sum error
PMF $N = 1000$, $K = 5$	237	512	2.2	1.85×10^{-1}	1.86×10^{-1}	7.1×10^{-6}
		1024	4.3	6.96×10^{-4}	6.96×10^{-4}	1.1×10^{-9}
		2048	8.6	3.80×10^{-11}	3.80×10^{-11}	2.3×10^{-13}
		4096	17.3	2.76×10^{-14}	2.76×10^{-14}	6.4×10^{-16}
LLR $N = 1000$, $K = 7$, $p_k \propto k^2$	399	1024	2.6	6.35×10^{-2}	6.36×10^{-2}	3.5×10^{-7}
		2048	5.1	1.63×10^{-4}	1.63×10^{-4}	1.2×10^{-10}
		4096	10.3	9.71×10^{-14}	9.70×10^{-14}	6.2×10^{-14}
LLR $N = 10^4$, $K = 10$	1989	4096	2.1	2.36×10^{-1}	2.37×10^{-1}	1.2×10^{-4}
		8192	4.1	8.96×10^{-4}	8.96×10^{-4}	3.7×10^{-10}
		16384	8.2	7.59×10^{-11}	7.60×10^{-11}	2.5×10^{-14}
		32768	16.5	3.82×10^{-13}	4.31×10^{-13}	2.9×10^{-14}
PMF $N = 10^4$, $K = 5$	2358	8192	3.5	6.78×10^{-3}	6.78×10^{-3}	7.0×10^{-7}
		16384	6.9	7.16×10^{-8}	7.16×10^{-8}	2.8×10^{-11}
		32768	13.9	4.43×10^{-13}	4.43×10^{-13}	1.3×10^{-15}

Table 1: The certificate against the aliasing actually present. Q is forced; the reference is the exact evaluation with $Q > \max(N, (K-1)N)$, over the first 256 frequencies. Uniform $x_0 = N/K$ and $p_k \propto k$ unless stated. The certificate reproduces the worst-case term aliasing to two or three digits (Lemma 1 is attained at $n = 0$), and the partial sum $\frac{1}{2}t_0 + \sum_{0<n<256} t_n$ is disturbed far less. Rows selected by `build_poisson_plan` under the 10^{-10} tolerance: 2048, 4096, 16384 and 32768 respectively.

7 Why the approximate de-Poissonization is not enough

The gamma search (`gamma_search.h`) uses the same Poissonized weights but never undoes the Poissonization: it minimizes $\log(\rho^{-N} \prod_k Z_k) + \log \mathsf{sinc}$, which omits even the normalization $P(T = N) \approx 1/\sqrt{2\pi V}$. That is harmless there, because γ only conditions the series — the tilt cancels exactly — so an $O(1/V)$ error in the objective merely moves γ slightly.

For the terms, $M_S(z_n)$ *is* the answer, and the natural approximation — the Gaussian (saddlepoint) evaluation of the circle integral, $[u^N]F \approx F(\rho)\rho^{-N}/\sqrt{2\pi V}$, i.e. keeping only the neighbourhood of $q = 0$ — fails twice. At $n = 0$ it carries the usual $O(1/V)$ relative error of a saddlepoint approximation (Daniels, 1954), straight into p. At larger n it fails outright: the phases $e^{-in\theta_k(i)}$ decorrelate the categories, the integrand is no longer concentrated near $q = 0$, and the Gaussian picture has nothing to approximate. Measured against the exact terms:

instance	γ	$\sqrt{V}$	relative error at $n = 0$	worst relative error, $n < 512$
LLR $N = 100$, $K = 20$, x_0 at the mode	0.137	10.8	9.7×10^{-4}	0.45
LLR $N = 1000$, $K = 7$, uniform x_0	0.996	399	3.7×10^{-2}	0.46
PMF $N = 1000$, $K = 5$, x_0 at the mode	0.287	37.4	1.2×10^{-4}	1.00

The exact version costs the Q-point sum instead of one point, and removes both errors.

8 An aside: selecting γ by the same Poissonisation

The gamma search just referred to (`multinomial/gamma_search.h`) is worth deriving in full, since it is the other place in this codebase where dropping the multinomial constraint turns an expensive

combinatorial object into a cheap independent one – and, unlike the terms, it gets away with never restoring the constraint at all.

`fill_gamma`'s job is to choose the real tilt γ that `fill_cells` bakes into every reward before the series runs. As `error_bounds.tex` establishes, the natural objective is the log-MGF of the *smoothed* statistic, $\Psi(\gamma) := \log M_{S+U}(\gamma) = \log M_S(\gamma) + \log \mathsf{sinc}(\pi\gamma/T)$, and its stationary point is the saddlepoint condition $E_\gamma[S+U] = 0$. The grid search that used to solve this (still available as `fill_gamma_grid`) evaluated Ψ exactly, via the same K-fold constrained convolution as the terms, at 128 points refined over several rounds. Poissonising Ψ turns each evaluation into the one-pass computation of §3: with $w_k(i;\gamma,\nu) = e^{\ell_k(i)+\nu i}$ now carrying an explicit γ inside $\ell_k(i)$ as well (recall $\ell_k(i) = \log p_k(i) - \gamma a_k(i)$ once `fill_cells` has run; here we treat γ as the free variable and undo that baking-in for the derivation) and $Z_k(\gamma,\nu) = \sum_i w_k(i;\gamma,\nu)$, define

$$A(\gamma,\nu) := \sum_k \log Z_k(\gamma,\nu), \qquad \Psi_{\mathrm{Pois}}(\gamma) := \min_\nu \big[A(\gamma,\nu) - \nu N\big] + \log N! + \log \mathsf{sinc}(\pi\gamma/T).$$

$A(\gamma,\cdot) - \nu N$ is convex in ν (a cumulant generating function minus a linear term), so its minimiser $\nu(\gamma)$ is exactly the constraint $\sum_k E_k[i] = N$ of §3 – the same tilt `gamma_moments` solves for by Newton bisection, warm-started from the previous γ's solution.

Proposition 5 (Ψ_{Pois} is convex, and its derivatives are free)**.** *Write $x = \pi\gamma/T$, $g = \log \mathsf{sinc}$, and let $E_k[\cdot]$, $\mathrm{Var}_k[\cdot]$, $\mathrm{Cov}_k(\cdot,\cdot)$ denote moments under $w_k(\cdot;\gamma,\nu(\gamma))/Z_k$. Then*

$$\Psi'_{\mathrm{Pois}}(\gamma) = -\sum_k E_k[a_k] + \tfrac{\pi}{T} g'(x), \qquad \Psi''_{\mathrm{Pois}}(\gamma) = \sum_k \mathrm{Var}_k[a_k] - \frac{\big(\sum_k \mathrm{Cov}_k(a_k,i)\big)^2}{\sum_k \mathrm{Var}_k[i]} + \Big(\tfrac{\pi}{T}\Big)^2 g''(x) \;\geq\; 0.$$

Proof. Write $V(\gamma) := \min_\nu[A(\gamma,\nu) - \nu N] = A(\gamma,\nu(\gamma)) - \nu(\gamma)N$. Since $\nu(\gamma)$ is an unconstrained interior minimiser, the envelope theorem gives $V'(\gamma) = \partial_\gamma A(\gamma,\nu(\gamma)) = \sum_k \partial_\gamma \log Z_k = -\sum_k E_k[a_k]$ (the $\nu'(\gamma)$ term drops out because $\partial_\nu[A - \nu N] = 0$ at $\nu(\gamma)$), which gives the first identity together with the sinc contribution.

For the second, differentiate the two first-order quantities $f_1(\gamma,\nu) := \sum_k E_k[i] - N$ and $f_2(\gamma,\nu) := -\sum_k E_k[a_k]$ along the implicit curve $\nu = \nu(\gamma)$ defined by $f_1 = 0$. Direct computation from $w_k = \exp(-\gamma a_k(i) + \log p_k(i) + \nu i)$ gives, for each k, $\partial_\gamma E_k[i] = -\mathrm{Cov}_k(a_k,i)$, $\partial_\nu E_k[i] = \mathrm{Var}_k[i]$, $\partial_\gamma E_k[a_k] = -\mathrm{Var}_k[a_k]$, $\partial_\nu E_k[a_k] = \mathrm{Cov}_k(a_k,i)$ (each a standard exponential-tilting identity: differentiating $\log Z_k$ once more brings down another centred factor). The implicit function theorem applied to $f_1 = 0$ gives $\nu'(\gamma) = -\partial_\gamma f_1 / \partial_\nu f_1 = \sum_k \mathrm{Cov}_k(a_k,i) / \sum_k \mathrm{Var}_k[i]$, and then

$$\Psi''_{\mathrm{Pois}}(\gamma) - \Big(\tfrac{\pi}{T}\Big)^2 g''(x) = \frac{d}{d\gamma} f_2(\gamma,\nu(\gamma)) = \partial_\gamma f_2 + \partial_\nu f_2 \cdot \nu'(\gamma) = \sum_k \mathrm{Var}_k[a_k] - \frac{\big(\sum_k \mathrm{Cov}_k(a_k,i)\big)^2}{\sum_k \mathrm{Var}_k[i]}.$$

Writing $R = \sum_k a_k(I_k)$ and $I = \sum_k I_k$ for the (independent, under the Poissonised measure) category totals, this right-hand side is exactly $\mathrm{Var}(R) - \mathrm{Cov}(R,I)^2/\mathrm{Var}(I)$, the residual variance $\min_\beta \mathrm{Var}(R - \beta I)$ of the best linear predictor of R from I – non-negative because a variance is. `sinc`'s own log is convex on $\mathbb{R}$ (it is $-\log$ of a normalised completely monotone kernel), so $g'' \geq 0$ too, and $\Psi''_{\mathrm{Pois}} \geq 0$ follows. □

This is a compressed restatement, in the Poissonised setting, of the general fact underlying every saddlepoint method: cumulant generating functions are convex, and their derivatives are moments. What makes it useful here is that both moments in Proposition 5 come out of *the same single pass* over the $K(N+1)$ cells that `gamma_moments` already runs to solve for $\nu(\gamma)$ – accumulating $\sum_i x w_k(i)$, $\sum_i x^2 w_k(i)$ for $x \in \{i, a_k(i)\}$ and their cross term costs one extra multiply-add per cell over the moment ν already needed, so the derivative is free in the sense that matters: no second evaluation of the objective, no finite-difference step size to choose.

The search. `gamma_newton_search` is a safeguarded Newton iteration (Newton where it behaves, bisection where it does not, in the style of Numerical Recipes' `rtsafe`) on Ψ'_{Pois} over the caller's bracket, unchanged at $[-2, 2]$. A subtlety: the step size cannot be the stopping test here, because Ψ''_{Pois} spans several orders of magnitude across the bracket – the sinc term alone contributes $(\pi/T)^2 g''(0) = (\pi/T)^2/12$ near $\gamma = 0$ but only $\sim 1/\gamma^2$ once $\pi\gamma/T$ exceeds one – so plain Newton can take steps far smaller than the remaining distance to the root without indicating it should stop. Only the width of the bracket is used to decide that, exactly as `eps_gamma` already specifies. Once the bracket has shrunk below `eps_gamma`, one further safeguarded Newton refinement of the last iterate is applied (clipped to that final bracket): it is accurate to the square of the bracket's own error, essentially for free, and matters because the series length is acutely sensitive to exactly where in a narrow bracket γ lands.

Why the geometric shortcut used for the terms does not apply here. The frequency loop of §9 advances $e^{-in\theta_k(i)}$ by a constant complex factor because that phase has no data-dependent rescaling in it. Every sum inside `gamma_moments` (and, before it, the FFT-tree's own `find_nu_gamma_vec`) is instead a log-sum-exp whose numerically stabilising pivot (`max_exp`) is chosen per γ – and per SIMD lane, when several γ are evaluated together. The exponents actually formed are $w_k(i)/e^{\text{pivot}(\gamma)}$ rather than $w_k(i)$ itself, and that data-dependent shift destroys the geometric relation between the exponentials at neighbouring γ. The fix is the one above – evaluate γ fewer times – not to make each evaluation itself incremental.

Measured. On the instances of Table 2 and a few others, at `eps_rel`$= 10^{-6}$:

N	K	stat	γ, grid $\to$ Newton	evaluations	time, grid $\to$ Newton (ms)	speedup	$\lvert\Delta p\rvert/p$
1000	7	LLR	0.997229 $\to$ 0.996335	128 $\to$ 6	53.1 $\to$ 4.0	13×	6.2×10^{-9}
100	20	LLR	0.993197 $\to$ 0.993762	128 $\to$ 6	16.9 $\to$ 1.2	15×	4.0×10^{-12}
100	20	LLR	0.147644 $\to$ 0.137001	128 $\to$ 8	34.6 $\to$ 1.4	25×	3.3×10^{-7}
1000	5	LLR	0.985135 $\to$ 0.982263	128 $\to$ 7	66.1 $\to$ 3.1	21×	9.7×10^{-6}
1000	5	PMF	0.334089 $\to$ 0.286481	128 $\to$ 7	42.3 $\to$ 2.5	17×	6.9×10^{-5}
500	50	LLR	0.831948 $\to$ 0.827904	128 $\to$ 6	182.5 $\to$ 13.3	14×	3.4×10^{-14}
2000	5	LLR	0.993197 $\to$ 0.991124	128 $\to$ 7	83.0 $\to$ 6.0	14×	6.3×10^{-6}

Both searches disagree on γ itself by 5×10^{-4}–5×10^{-2} (the surrogate's saddlepoint is exact only to $O(1/N)$), yet the resulting p-values agree to 10^{-14}–10^{-5}: as §7 already notes, γ only conditions the series – the tilt cancels out of p exactly – so this level of disagreement is expected and harmless. What is not free is the time: 13–25× per instance here, and 12× aggregated over a sweep of 747 instances spanning $N \in [20, 1000]$, $K \in [2, 20]$ and both statistics in earlier measurement.

9 The evaluator

The implementation is `src/multinomial/fourier_poisson.h`, selected by `options->poisson = 1`, which overrides `fft_precision` and `use_fft_precompute` for the series terms and never builds the merge-factor plan.

Plan (`poisson_plan_t`, built once per `series()`): the tilt ν and variance V; the normalized weights $\pi_k(i)$ for all $K(N+1)$ cells; Q with its certificate; the bit-reversed destination of each count $i \bmod Q$; the $Q/2$ roots; the extractor ω^{qN}/Q in the code's sign convention; and the magnitude `resN_exp` $= \exp(\log N! + \sum_k \log Z_k - \nu N)$ in quad precision.

Per frequency vector. As in the other evaluators, one pass carries `STRIDE` $= 8$ frequencies in SIMD lanes. For each category: zero $2Q$ lanes of bins; fold $\pi_k(i)e^{-in\theta_k(i)}$ into the bit-reversed bin of $i \bmod Q$, with the phase produced by the recurrence of `fourier.h` (per-thread `phase` times shared `lane`), so no transcendental function is evaluated in the loop; run the iterative FFT, whose input is already permuted and whose first level uses root one only; and multiply the result into the running product. Finally one dot product with the extractor gives the unit of (3), multiplied by `resN_exp` and $\mathsf{sinc}(z_n)$ as in every other evaluator. Every lane shares the fold positions, the roots and the product structure, so every operation is a whole-vector one.

Cost and memory. Per frequency the work is

$$K\Big[(N+1) \;+\; \tfrac{Q}{2}\log_2 Q \;+\; Q\Big] \;+\; Q$$

elementary complex operations, against $K \cdot \mathrm{calls}(M)$ merges (1) for the tree, and there is a single forward transform per category instead of a descent and an ascent. The plan takes $8K(N+1) + O(N+Q)$ bytes and each thread $4Q$ vectors ($256Q$ bytes) of scratch, against $16K \cdot \mathrm{calls}(M)$ bytes and about $3M$ vectors per thread for the tree.

Threads. The evaluator is a drop-in thread function: thread t takes the frequencies $n \equiv t \pmod{\texttt{threads}}$, so that the phase recurrence has a constant step and survives chunk boundaries. The plan is read-only and shared. The deviations of Table 2 are the same to all printed digits at 8, 3 and 1 threads.

A persistent pool. `series()` calls whichever evaluator is selected once per chunk of `threads * vecs_per_thread * STRIDE` frequencies – typically some tens to a few hundred times per run, occasionally into the thousands for a slowly converging instance. The pre-existing dispatcher, `eval_fourier_parallel`, called `pthread_create` and `pthread_join` `threads` times on *every* call, which is unnecessary work repeated every chunk: nothing about a worker's job changes chunk to chunk except which frequencies it takes next. `fourier_pool_t` instead creates the worker threads once per `series()` call and reuses them for every chunk, through a textbook reusable barrier – a generation counter and two condition variables, one workers wait on for the next chunk and one the caller waits on for completion – so only the fields that actually change (`start`, `end`, `warm`) are touched between chunks; everything else (`mult`, `res_arr`, the plan, the phase tables, even the evaluator function pointer) is written into each worker's persistent `arguments_t` once, at pool creation.

In isolation (no evaluator work, just the synchronisation), waking 8 parked threads costs about $20\,\mu$s against $61\,\mu$s to create and join 8 fresh ones – a $3\times$ reduction in pure overhead. Against the real evaluators, that overhead is usually a small fraction of one chunk's actual work (a few percent at the instance sizes in Table 3, lost in run-to-run noise), but it dominates at the smallest instances, where a chunk's work is itself only tens of microseconds:

N	K	evaluator	threads	create/join (μs/chunk)	pool (μs/chunk)	speedup
20	2	Poisson	8	50.3	25.5	2.0×
20	3	Poisson	8	56.5	29.5	1.9×
50	3	Poisson	8	67.7	40.3	1.7×
20	2	tree	8	64.9	40.3	1.6×

The pool also removes the two `malloc`/`free` pairs (`arguments_t` and the `pthread_t` array) that `eval_fourier_parallel` repeated on every call; that cost is not visible in the timings above but

N	K	stat	x_0	γ	$\sqrt{V}$	Q	$D(Q)$	$\max_n \lvert\Delta t_n\rvert/t_0$	$\lvert\Delta p\rvert/p$
100	20	LLR	mode	0.137	10.8	128	8.7×10^{-15}	1.6×10^{-14}	1.1×10^{-14}
100	20	LLR	unif, k^5	0.994	35.1	512	6.1×10^{-15}	9.4×10^{-14}	9.5×10^{-14}
1000	7	LLR	unif, k^2	0.996	399.2	4096	9.7×10^{-14}	2.5×10^{-13}	2.6×10^{-13}
1000	7	PMF	unif, k^2	0.997	327.9	4096	3.1×10^{-14}	1.5×10^{-12}	1.5×10^{-12}
1000	5	PMF	mode	0.287	37.4	512	1.9×10^{-13}	9.4×10^{-14}	4.3×10^{-14}
1000	3	LLR	unif	0.984	247.4	2048	1.1×10^{-14}	3.9×10^{-13}	3.7×10^{-13}
1000	2	LLR	unif	0.985	232.6	1024	exact	1.9×10^{-13}	1.2×10^{-13}
500	50	LLR	unif	0.828	52.6	512	1.9×10^{-14}	2.1×10^{-13}	1.9×10^{-13}
50	3	LLR	unif, $k^{1/2}$	0.475	9.8	128	exact	7.7×10^{-15}	1.0×10^{-15}
5000	10	LLR	unif	0.995	994.1	8192	7.4×10^{-11}	7.5×10^{-11}	2.3×10^{-12}

Table 2: Poissonized evaluator against the FFT tree. Term deviations over the first 512 frequencies (256 at $K = 50$, 128 at $N = 5000$), identical at 8, 3 and 1 threads; p-values at the same fixed term budget with `eps_rel = 0` and `extrapolate = 0`. "unif" is $x_0 = N/K$, "mode" is $x_0 \approx N\mathbf{p}$, and k^s gives $p_k \propto k^s$ ($s = 1$ when omitted). "exact" marks $Q > \max(N, (K-1)N)$, where no alias exists and no certificate is needed.

disappears with the same change. At `threads`= 1 the pool skips its synchronisation machinery entirely and calls the evaluator directly, matching the previous sequential path.

10 Verification

Table 2 compares the new evaluator with the FFT tree on instances chosen to span both statistics, small and large K, the deep tail and moderate p-values, and Q from 128 to 8192. Term by term the two agree to within the aliasing tolerance and rounding; the maximum deviation is the same to all printed digits at 8, 3 and 1 threads. At a fixed term budget, with stopping and extrapolation disabled, the p-values agree to between 10^{-15} and 2.3×10^{-12} relative. The one row where the term deviation reaches 7.5×10^{-11} is the one whose Q was accepted at a certificate of 7.4×10^{-11}, exactly as Lemma 1 predicts, and its p-value still agrees to 2.3×10^{-12}.

11 The statistic's interval, without the $O(KN^2)$ recursion

Before the series can run, `fill_interval` must locate the range $[s_{\min}, s_{\max}]$ of $S(x) - S(x_0) = \sum_k g_k(x_k) - g_k(x_{0,k})$ over integer $x \geq 0$, $\sum_k x_k = N$, where g_k is the category-k contribution to whichever statistic is in use (the LLR, a general power-divergence, or the pmf log-density). It did this with a $K(N{+}1)$-cell min/max-plus convolution, the combinatorial analogue of the tree evaluator's own recursion – $O(KN^2)$, and, at $N = 10^4$, $K = 50$, the single largest item in setup (§12 below). Both endpoints admit a much cheaper closed form.

Proposition 6 (g_k is concave)**.** *For every real λ (the power-divergence family, LLR at $\lambda = 0$), $g_k(x) = -\frac{1}{\lambda(\lambda+1)} x\big((x/Np_k)^\lambda - 1\big)$ is strictly concave on $x > 0$:*

$$g_k''(x) = -(Np_k)^{-\lambda} x^{\lambda-1} < 0.$$

The LLR limit $\lambda \to 0$, $g_k(x) = -x \log(x/Np_k)$, has $g_k''(x) = -1/x < 0$ directly. For the pmf statistic, $g_k(x) = x \log p_k - \log\Gamma(x{+}1)$ is concave because $\log\Gamma$ is convex on $(0, \infty)$ — the defining property, by the Bohr–Mollerup theorem, that makes Γ the unique log-convex extension of the factorial to the reals — so g_k, an affine function of x minus a convex one, is concave.

Proof. Differentiate the power-divergence case twice; the factor $\lambda(\lambda+1)$ introduced by the prefactor cancels exactly against the one produced by differentiating $x^{\lambda+1}$ twice, so the sign is uniformly negative with no special case at $\lambda \in (-1, 0)$. The pmf case needs no derivative at all: log-convexity of Γ on $(0, \infty)$ is exactly the Bohr–Mollerup characterization of it. □

Proposition 7 (exact minimum, $O(K)$)**.** *$S(x) = \sum_k g_k(x_k)$, being concave and separable, attains its minimum over the polytope $\{x \geq 0, \sum x_k = N\}$ exactly at one of its K vertices Ne_k ("all N observations in category k"): $s_{\min} = \min_k g_k(N) - r_0$, where $r_0 = S(x_0)$.*

Proof. Writing any x in the polytope as the convex combination $x = \sum_k (x_k/N)(Ne_k)$ of the vertices and applying concavity, $S(x) \geq \sum_k (x_k/N) S(Ne_k) \geq \min_k S(Ne_k)$; the vertices are themselves feasible, so the minimum over the whole polytope equals the minimum over just the K vertices. $S(Ne_k) = g_k(N) + \sum_{j \neq k} g_j(0) = g_k(N)$ since $g_j(0) = 0$. Shifting by x_0, the vertex value of $S(x) - S(x_0)$ is $g_k(N) - r_0$, and both $g_k(N)$ and r_0 are already available from `mult->matrix` and `mult->r0` – no new evaluation of the reward function is needed. □

Proposition 8 (maximum, power-divergence family: closed-form bound, $O(1)$)**.** *For the power-divergence family (any real λ, LLR included), $s_{\max} \leq -r_0$, with equality of the* continuous *relaxation (dropping integrality on x).*

Proof. Substituting $x_k = Np_k$ into g_k' gives $g_k'(Np_k) = -1/(\lambda + 1)$ for every k – the same value regardless of p_k – so $x = Np$ satisfies the stationarity condition of the equality-constrained Lagrangian with a single shared multiplier, and by concavity (Proposition 6) that stationary point is the global maximum of the continuous relaxation. Since the base of the power ratio is 1 there, $g_k(Np_k) = 0$ for every k, so the continuous maximum of S is exactly 0 and of $S(x) - S(x_0)$ is exactly $-r_0$. The true, integer-constrained maximum can only be smaller, since the integer lattice is a subset of the relaxed domain. (This is the familiar fact that a power-divergence statistic is non-negative and vanishes only when the observation matches its expectation – no search is needed to find it.) □

Maximum, pmf statistic: GreedyModeFind, $O(K \log K)$. Proposition 8 does not apply to the pmf statistic: $g_k'(x) = \log p_k - \psi(x+1)$ has no closed-form inverse, and $x = Np$ is not in general a stationary point of $\sum_k g_k$ (the pmf is not itself a divergence normalised to vanish at the mean, only asymptotically so). But $S(x) = \log P(x) - \log N!$, so maximizing S over the integer polytope is exactly the problem of finding a *mode* of Multinomial(N, p) – and White and Hendy (2010)'s `GreedyModeFind` solves that exactly in $O(K \log K)$. It starts from the unique mode $w_k = \lfloor ap_k \rfloor$ of a nearby, trivially computed distribution (their Theorem 1, for $a = N + K/2$), then walks frequency by frequency towards N; at each step, incrementing (or decrementing) whichever category k maximises $p_k/(x_k+1)$ (resp. x_k/p_k) strictly increases $P(x)$ and preserves optimality among reachable vectors (their Theorems 2–3). A binary max-heap keyed on that per-category score turns "pick the best category, update its score" into an $O(\log K)$ operation, for $O(K \log K)$ total (we use only their single-mode algorithm, `GreedyModeFind1`; the tie-tracking needed to enumerate the full set of joint modes is unnecessary here, since only the value $S(x^*)$ at one mode x^* is wanted). $s_{\max} = S(x^*) - r_0$ is then exact, not merely a bound.

Measured. Both formulas were checked against `fill_interval`'s DP directly (not just through the series) across uniform, skewed ($p_k \propto k^2$), and deliberately dominant-category ($p_1 \gg p_{k>1}$) distributions, N up to 500, K up to 50, and $\lambda \in \{-0.5, -0.3, 0, 2/3, 1, 2\}$ plus the pmf statistic: $s_{\min}$ agrees with the DP to floating-point noise ($< 10^{-8}$ relative) in every case, including the $\lambda \neq 0$

family's exact-in-theory formula and the pmf statistic's GreedyModeFind, and $s_{\max}$ for the power-divergence family is always a valid upper bound (never below the DP's exact value), with slack small relative to $|s_{\min}|$ – consistent with $s_{\min}$ being the binding constraint on $W = \max(s_{\max}, -s_{\min})$ in every case checked, including the adversarial dominant-category distributions. Isolated from the rest of setup, at $K = 100$: the DP's time scales as expected with N (about 4× per doubling, from 0.10 s at $N = 2000$ to 9.1 s at $N = 20{,}000$ for the pmf statistic), while GreedyModeFind finishes in about 2 μs regardless of N – roughly five orders of magnitude faster at $N = 20{,}000$. §12 below reports the effect on the whole pipeline.

12 Measured speedup

All timings are on an Apple M5 (4 performance and 6 efficiency cores) with `gcc-15 -Ofast -march=native`, `STRIDE` = 8 lanes through SIMDe's AVX-512 emulation on NEON, the best of three runs, and uniform $x_0 = N/K$ with $p_k \propto k$ — the shape of the benchmark suite, and the deep tail — unless stated.

12.1 Per frequency

Table 3 isolates the evaluators: the cost of one frequency, excluding plan construction, at one thread and at the default eight.

Three patterns stand out.

- **Deep tail with $Q = M$: about** 3× (2.4–4.0×). With the same transform size the new evaluator is still faster, because a butterfly is cheaper than a merge and there is one pass per category instead of a descent and an ascent.
- **Each halving of Q roughly doubles the gain**: 4–9× at $Q = M/2$ and, at $Q = M/4$ ($N = 10^4$, $K = 50$), 11× at one thread widening to 20× at eight. Because Q moves in powers of two, similar instances can land a factor of two apart: LLR and PMF at $N = 10^4$, $K = 10$ have $\sqrt{V}$ within 3% of each other, but the certificate accepts $Q = 16384$ for LLR and not for PMF, whose geometric per-category laws have the heavier tails.
- **Away from the tail**, $Q \ll M$ and the fold dominates: 7–21× at one thread, 5–20× at eight.

At $N = 100$ the eight-thread ratios are still somewhat compressed relative to one thread (e.g. 2.7× against 3.0× for LLR $K = 3$) because at 1–4 μs per frequency, even the persistent pool's synchronisation cost (§9, about 2.5 μs per chunk at 8 threads) is not entirely negligible next to the work; both evaluators pay it, and it is far smaller than the ∼8 μs per chunk the old per-chunk `pthread_create/join` cost here.

At $N = 10^4$, $K = 50$ the pattern reverses: the tree/Poisson speedup is *larger*, not smaller, at eight threads than at one (19.9× against 11.1× for LLR, 19.8× against 11.1× for PMF) – the widest thread-count gap in the table, and the opposite of the small-instance compression just above. The likely reason is the per-thread footprint gap of §12.2: 12.5 MB of scratch per thread for the tree at $N = 10^4$ against 2–8 MB for the Poissonized evaluator, so eight tree threads compete for memory bandwidth by the time eight Poissonized ones do not, and the tree's parallel scaling suffers exactly where its plan is largest.

					1 thread (μs)			8 threads (μs)		
N	K	stat	$\sqrt{V}$	Q / M	tree	Poisson	speedup	tree	Poisson	speedup
100	3	LLR	25.4	256 / 256	9.7	3.2	3.0×	2.9	1.1	2.7×
100	5	LLR	24.4	256 / 256	16.5	5.2	3.2×	4.2	1.7	2.6×
100	10	LLR	20.2	256 / 256	32.8	9.9	3.3×	7.0	2.9	2.4×
100	20	LLR	15.7	128 / 256	66.3	9.6	6.9×	13.6	2.8	4.8×
100	50	LLR	11.1	128 / 256	165.4	23.7	7.0×	33.4	5.0	6.7×
100	3	PMF	26.1	256 / 256	10.0	3.3	3.1×	2.9	1.1	2.7×
100	5	PMF	25.1	256 / 256	16.6	5.3	3.2×	4.3	1.7	2.6×
100	10	PMF	20.9	256 / 256	33.2	10.2	3.3×	7.1	2.9	2.4×
100	20	PMF	16.8	256 / 256	66.5	20.0	3.3×	13.6	4.8	2.9×
100	50	PMF	12.9	128 / 256	166.0	23.9	6.9×	34.0	5.0	6.8×
1000	3	LLR	247.4	2048 / 2048	108.8	39.4	2.8×	22.5	7.8	2.9×
1000	5	LLR	236.5	2048 / 2048	181.6	63.9	2.8×	36.9	12.3	3.0×
1000	10	LLR	198.8	2048 / 2048	361.6	124.5	2.9×	72.9	23.6	3.1×
1000	20	LLR	155.5	2048 / 2048	724.0	247.1	2.9×	145.8	46.1	3.2×
1000	50	LLR	105.5	1024 / 2048	1804.7	286.1	6.3×	359.2	55.4	6.5×
1000	3	PMF	250.4	2048 / 2048	108.8	39.2	2.8×	22.4	7.9	2.8×
1000	5	PMF	237.0	2048 / 2048	181.1	63.6	2.8×	36.9	12.3	3.0×
1000	10	PMF	195.0	2048 / 2048	361.4	124.3	2.9×	73.5	23.9	3.1×
1000	20	PMF	150.4	2048 / 2048	720.4	246.8	2.9×	145.2	46.2	3.1×
1000	50	PMF	102.1	1024 / 2048	1808.2	286.2	6.3×	354.8	55.4	6.4×
10^4	3	LLR	2471	32768 / 32768	2251.8	775.6	2.9×	816.4	209.0	3.9×
10^4	5	LLR	2362	32768 / 32768	3805.0	1289.3	3.0×	1314.8	335.8	3.9×
10^4	10	LLR	1989	16384 / 32768	7636.2	1303.7	5.9×	2600.2	277.1	9.4×
10^4	20	LLR	1558	16384 / 32768	15288.7	2597.5	5.9×	5242.5	553.4	9.5×
10^4	50	LLR	1060	8192 / 32768	38406.6	3451.0	11.1×	13277.9	668.0	19.9×
10^4	3	PMF	2495	32768 / 32768	2298.4	803.3	2.9×	800.7	225.6	3.6×
10^4	5	PMF	2358	32768 / 32768	3821.7	1320.1	2.9×	1339.6	353.8	3.8×
10^4	10	PMF	1937	32768 / 32768	7648.2	2559.5	3.0×	2673.8	664.6	4.0×
10^4	20	PMF	1488	16384 / 32768	15319.4	2567.9	6.0×	5317.0	571.8	9.3×
10^4	50	PMF	997	8192 / 32768	38235.0	3452.7	11.1×	13513.3	683.7	19.8×
moderate p-values, x_0 at the mode										
100	20	LLR	10.8	128 / 256	66.4	9.7	6.9×	13.7	2.8	4.8×
1000	5	PMF	37.4	512 / 2048	180.7	15.1	12.0×	36.9	4.0	9.3×
1000	20	LLR	33.2	256 / 2048	721.0	34.5	20.9×	146.0	7.3	20.1×

Table 3: Cost of one series term, FFT tree (precompute plan) against the Poissonized evaluator, both dispatched through the persistent thread pool of §9 – the same path `series()` actually uses.

12.2 Memory and plan construction

		plan (MB)		plan build (ms)	
N	K	tree	Poisson	tree	Poisson
1000	50	27.3	0.41	5.3	1.4
10^4	3	35.2	1.06	6.9	3.4
10^4	10	117.5	1.21	21.2	7.0
10^4	20	235.0	1.98	41.8	12.5
10^4	50	587.5	4.08	115.3	15.0

Per-thread scratch is 12.5 MB for the tree at $N = 10^4$ against $256Q$ bytes, 2–8 MB, for the Poissonized evaluator. The Poisson plan build includes the certificate's scalar evaluations.

12.3 End to end

Finally the whole pipeline — setup, gamma search, plan and adaptive series, now through the persistent thread pool of §9, the robust stopping rule of §13 and the $O(K)/O(K \log K)$ statistic interval of §11 in place of `fill_interval`'s $O(KN^2)$ recursion — with the default options (`extrapolate = 2`), `max_iter = 100000` and eight threads, at `eps_rel` of 10^{-3} and 10^{-6}: 72 runs over $N \in \{100, 1000, 5000, 10^4\}$. Across all of them the series time *per term* improves by a factor between 2.0 and 22.1, with median 3.0. The stopping rule of §13 makes the tree and Poisson evaluators report the *same* term count in 71 of the 72 runs (the one exception differs by 6 terms out of 670). A selection, with speedups of the series alone and of the whole run:

N	K	stat	`eps_rel`	setup (s)	series, tree → Poisson (s)	series	total	terms
1000	5	LLR	10^{-3}	0.0024	0.0353 → 0.0126	2.8×	2.5×	888
1000	50	LLR	10^{-6}	0.0221	0.1478 → 0.0266	5.6×	3.4×	332
5000	5	LLR	10^{-6}	0.0200	7.913 → 2.364	3.4×	3.3×	14915
5000	10	LLR	10^{-6}	0.0354	0.818 → 0.130	6.3×	5.0×	783
5000	50	PMF	10^{-6}	0.177	2.165 → 0.142	15.3×	6.4×	327
10^4	5	LLR	10^{-3}	0.0384	0.834 → 0.276	3.0×	2.7×	546
10^4	20	LLR	10^{-3}	0.183	1.274 → 0.164	7.8×	3.8×	173
10^4	50	LLR	10^{-6}	0.308	7.298 → 0.330	22.1×	10.3×	333
1000	5	PMF, mode	10^{-6}	0.0030	0.540 → 0.060	9.0×	8.6×	12025
1000	20	LLR, mode	10^{-3}	0.0096	0.203 → 0.013	15.7×	9.1×	1164

Term counts are given once per row since the two evaluators now agree exactly in every row shown. Setup is now dominated by `fill_rewards` ($O(KN)$: filling the $K(N+1)$-cell reward matrix) and the gamma search, rather than by locating the statistic's range: replacing `fill_interval`'s $O(KN^2)$ recursion with the closed-form bound and GreedyModeFind of §11 cuts setup at $N = 10^4, K = 50$ from 1.45 s to 0.31 s, and at $N = 5000, K = 50$ (PMF) from 0.75 s to 0.18 s. With setup no longer the larger cost even at $N = 10^4$, the end-to-end gain across the table now tracks the series speedup much more closely: 1.7–14.6× over the full 72-run sweep, against 2.1–4.4× when `fill_interval` was still the setup path.

Some instances at `eps_rel` $= 10^{-6}$ genuinely do not converge within the 100,000-term budget under either stopping criterion – LLR and PMF at $K = 3$, $x_0 = N/K$, are the clearest cases in this sweep. This is not a regression from either fix: re-running the plain term-ratio rule alone (`extrapolate`= 0) on the same instances with the cap raised to 200,000 terms shows it does not converge there either: the slow convergence is a property of these instances, not of either stopping criterion.

13 Robust stopping for `extrapolate` ≥ 2

`extrapolate` ≥ 2 stops the series on either of two independent conditions, whichever is satisfied first: the pre-existing term-ratio rule (`CONV_COUNT` consecutive terms with $|\text{term}_n/p_{n-1}| <$ `eps_rel`/`CONV_COUNT`), or a check on a second, shallow extrapolation of the same partial sums.

That second extrapolation runs the same DQELG algorithm as the reporting one (§9), but with a much shallower table (depth 8 rather than 50 – deep columns are exactly where catastrophic cancellation lives), and its raw output is smoothed by a running median of the last 5 values. Convergence is a question about that smoothed sequence: has it stayed within `eps_rel` of itself over a trailing window of max(`CONV_COUNT`, $i/2$) terms. A single outlier term cannot flip this test, because by the time it would matter it has aged out of both the median and the window. When this check is the one that stops the series, `pval` and `err_rel_ext` are also drawn from this same smoothed statistic and its trailing-window spread, rather than from the deep reporting table – the value reported is always the one that justified stopping.

Stability under perturbation. Repeating every run in this section with every term's magnitude perturbed by -1, 0 and $+1$ ulp of `lgac[N]` (a relative change of 10^{-13}–10^{-12}), at `eps_rel`$= 10^{-6}$:

instance	evaluator (threads)	terms (-1/0/$+1$ ulp)	$\lvert\Delta p\rvert/p$ (-1,$+1$ vs. 0)
LLR $N = 1000$, $K = 3$	tree (8)	200000 / 200000 / 200000	9×10^{-13}, 9×10^{-13}
PMF $N = 1000$, $K = 5$	tree (8)	15341 / 15341 / 15341	9×10^{-13}, 8×10^{-12}
LLR $N = 100$, $K = 5$	tree (8)	106249 / 106249 / 106249	6×10^{-14}, 6×10^{-14}
PMF $N = 100$, $K = 3$	tree (8)	200000 / 200000 / 200000	6×10^{-14}, 6×10^{-14}
PMF $N = 1000$, $K = 5$	Poisson (8)	15341 / 15341 / 15341	5×10^{-10}, 6×10^{-9}
PMF $N = 1000$, $K = 5$	Poisson (3)	15341 / 15341 / 15341	3×10^{-11}, 2×10^{-10}
PMF $N = 1000$, $K = 5$	Poisson (1)	15341 / 15341 / 15341	7×10^{-12}, 9×10^{-13}
LLR $N = 1000$, $K = 3$	Poisson (1)	200000 / 200000 / 200000	9×10^{-13}, 9×10^{-13}

Every instance gives the same term count for all three perturbations, at every evaluator and thread count tried, and the reported p agrees to 10^{-9}–10^{-13} relative – the size of the perturbation itself, not an amplified reaction to it.

The window check firing. On LLR, $N = 1000$, $K = 7$, $p_k \propto k^2$, `eps_rel` $= 10^{-3}$, the window check stops the series at 92 terms, well before the term-ratio rule would (which needs 224). The result is still trustworthy: 4.4×10^{-5} relative error against a 65536-term reference, within the requested 10^{-3}, and `err_rel_ext`$= 8.6 \times 10^{-4}$ correctly reports a wider margin than the 224-term run's 2.0×10^{-6} would. On LLR, $N = 1000$, $K = 20$, x_0 at the mode, `eps_rel` $= 10^{-3}$, the same 1164-term stopping point is reproduced under all three perturbations, and the reported p agrees to 9×10^{-13} across them, honestly labelled by a larger `err_rel_ext` of 3–8 $\times 10^{-2}$ in this instance – a case where `eps_rel` $= 10^{-3}$ is optimistic for the number of terms actually available, and the diagnostic says so rather than reporting a falsely precise-looking number.

14 Summary

- Dropping the multinomial constraint makes the K categories independent; a Q-point trapezoid rule for Cauchy's integral on the saddlepoint circle restores it exactly up to aliasing (Proposition 2).

- The aliasing of every term is bounded by the aliasing at frequency zero (Lemma 1), and that is measured exactly by comparing Q with $2Q$ (Proposition 3). The measurement is a certificate for the probability-mass statistic (Proposition 4) and matches the true aliasing to two or three digits in every measured case, including LLR.
- Q scales with the spread of the tilted total count: about $10\sqrt{N}$ away from the tail, and about $N/\sqrt{K}$ in the deep tail, where the tilt makes the per-category laws geometric.
- Against the FFT tree: p agrees to 10^{-15}–2.3×10^{-12}; a single term is about 3× cheaper in the deep tail when $Q = M$, 4–20× when Q is $M/2$ or $M/4$, and 4–22× away from the tail; the plan is 30–140× smaller.
- The evaluator is opt-in (`options->poisson`); it never builds the tree's plan.
- The same construction selects γ: the Poissonized log-MGF is convex, its derivatives come from the envelope theorem out of the moment pass already run for the tilt ν (Proposition 5), and a safeguarded Newton iteration on it is 13–25× faster than the 128-point grid it replaces, per instance measured.
- A persistent thread pool replaces per-chunk thread creation in `series()`: up to 2× at the smallest instances, where synchronisation overhead is otherwise comparable to the work itself; negligible where the per-chunk work already dominates.
- The `extrapolate` ≥ 2 stopping rule combines the term-ratio rule with a check on a shallow, median-smoothed extrapolation's trailing-window stability, and reports from that same robust statistic whenever it is the one that stops the series; verified stable to 10^{-9}–10^{-13} under a one-ulp perturbation of every term, across both evaluators and every thread count tried, and confirmed to stop a genuinely ringing instance early (224 → 92 terms) without loss of accuracy (§13).
- Locating the statistic's range, previously an $O(KN^2)$ dynamic program, is now $O(K)$ for the power-divergence family (concavity gives an exact vertex-based minimum and a closed-form maximum bound) and $O(K \log K)$ for the probability-mass statistic (an exact maximum via White and Hendy's multinomial-mode algorithm), verified against the DP across skewed and dominant-category distributions and cutting setup time by 4–5× at $N = 10^4$ (§11).

References

F. Bornemann. Accuracy and stability of computing high-order derivatives of analytic functions by Cauchy integrals. *Foundations of Computational Mathematics*, 11(1):1–63, 2011.

H. E. Daniels. Saddlepoint approximations in statistics. *Annals of Mathematical Statistics*, 25(4):631–650, 1954.

S. G. Hoggar. Chromatic polynomials and logarithmic concavity. *Journal of Combinatorial Theory, Series B*, 16(3):248–254, 1974.

P. Jacquet and W. Szpankowski. Analytical depoissonization and its applications. *Theoretical Computer Science*, 201(1–2):1–62, 1998.

B. Levin. A representation for multinomial cumulative distribution functions. *The Annals of Statistics*, 9(5):1123–1126, 1981.

J. N. Lyness and C. B. Moler. Numerical differentiation of analytic functions. *SIAM Journal on Numerical Analysis*, 4(2):202–210, 1967.

L. N. Trefethen and J. A. C. Weideman. The exponentially convergent trapezoidal rule. *SIAM Review*, 56(3):385–458, 2014.

W. T. J. White and M. D. Hendy. A fast and simple algorithm for finding the modes of a multinomial distribution. *Statistics and Probability Letters*, 80(1–2):63–68, 2010.